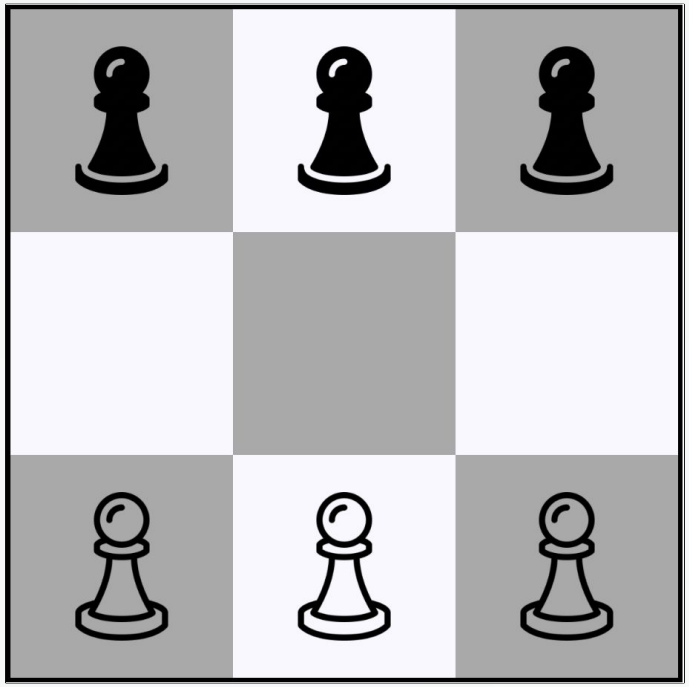
52405 DUBUC Julien

2023-2024

Développement d'une IA pour maîtriser le jeu Hexapawn



https://hexapawn-ml.netlify.app/

Sommaire

I) Présentation du jeu

Plateau

Règles

Fonctions

Exemple

II) Apprentissage par renforcement

Explication/Fonctions

Exemple

Augmentation du plateau

III) Création d’un arbre de jeu

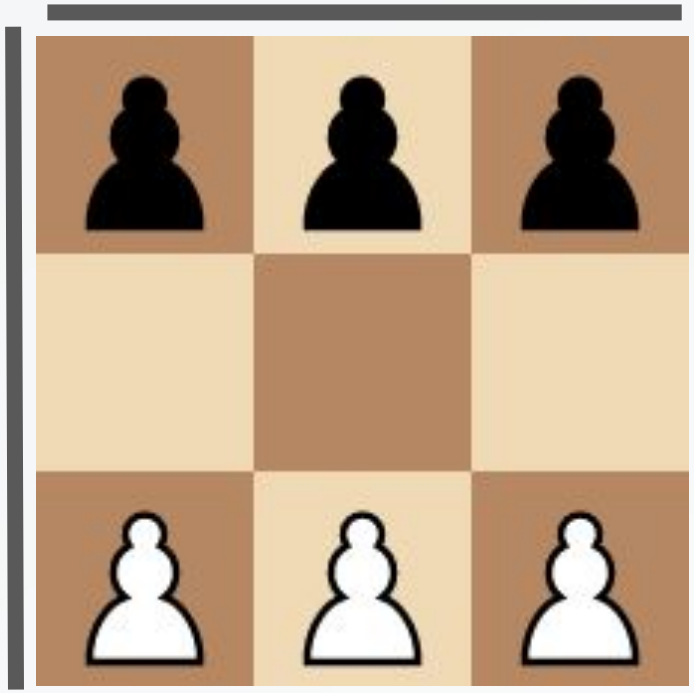
Explication/Fonctions

Conclusion

Comparaison/Limites

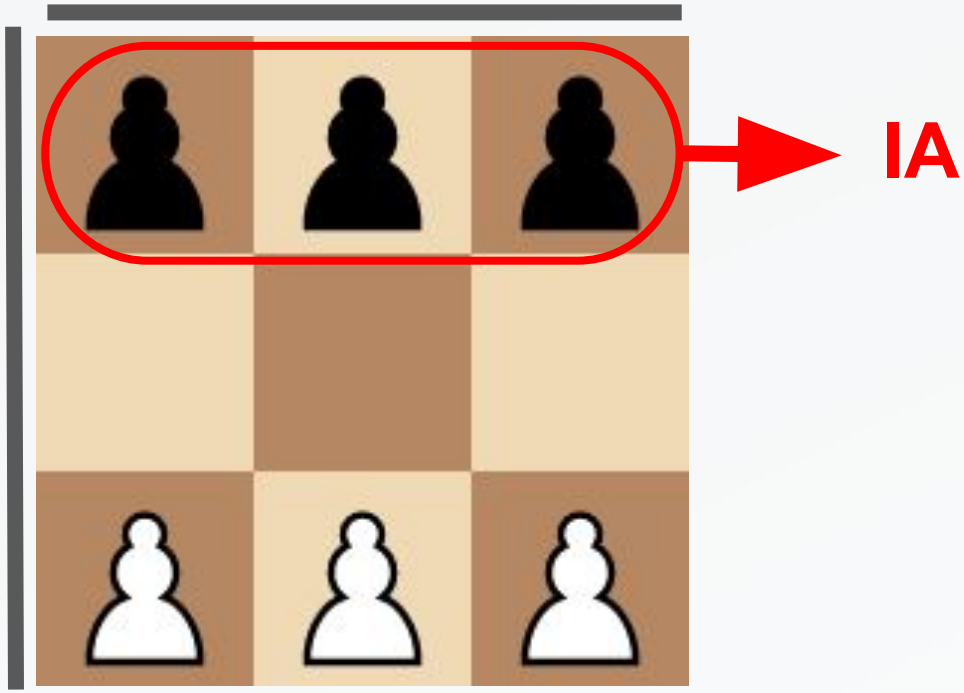
Plateau

Taille : 3x3



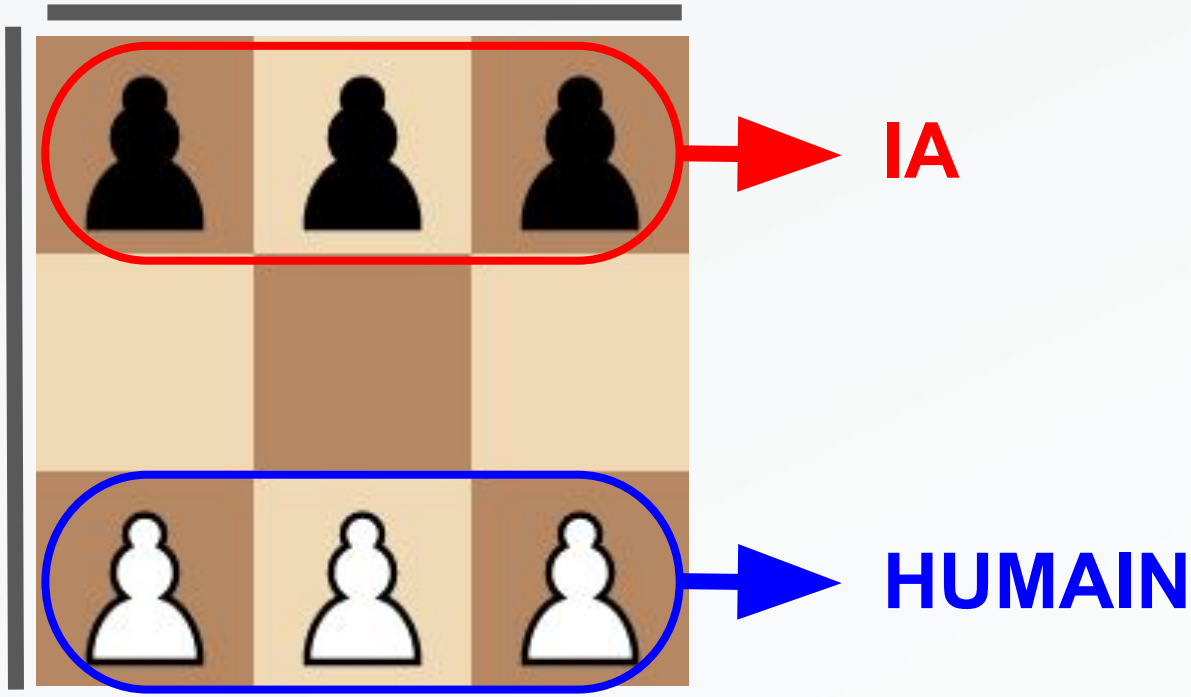
Plateau

Taille : 3x3



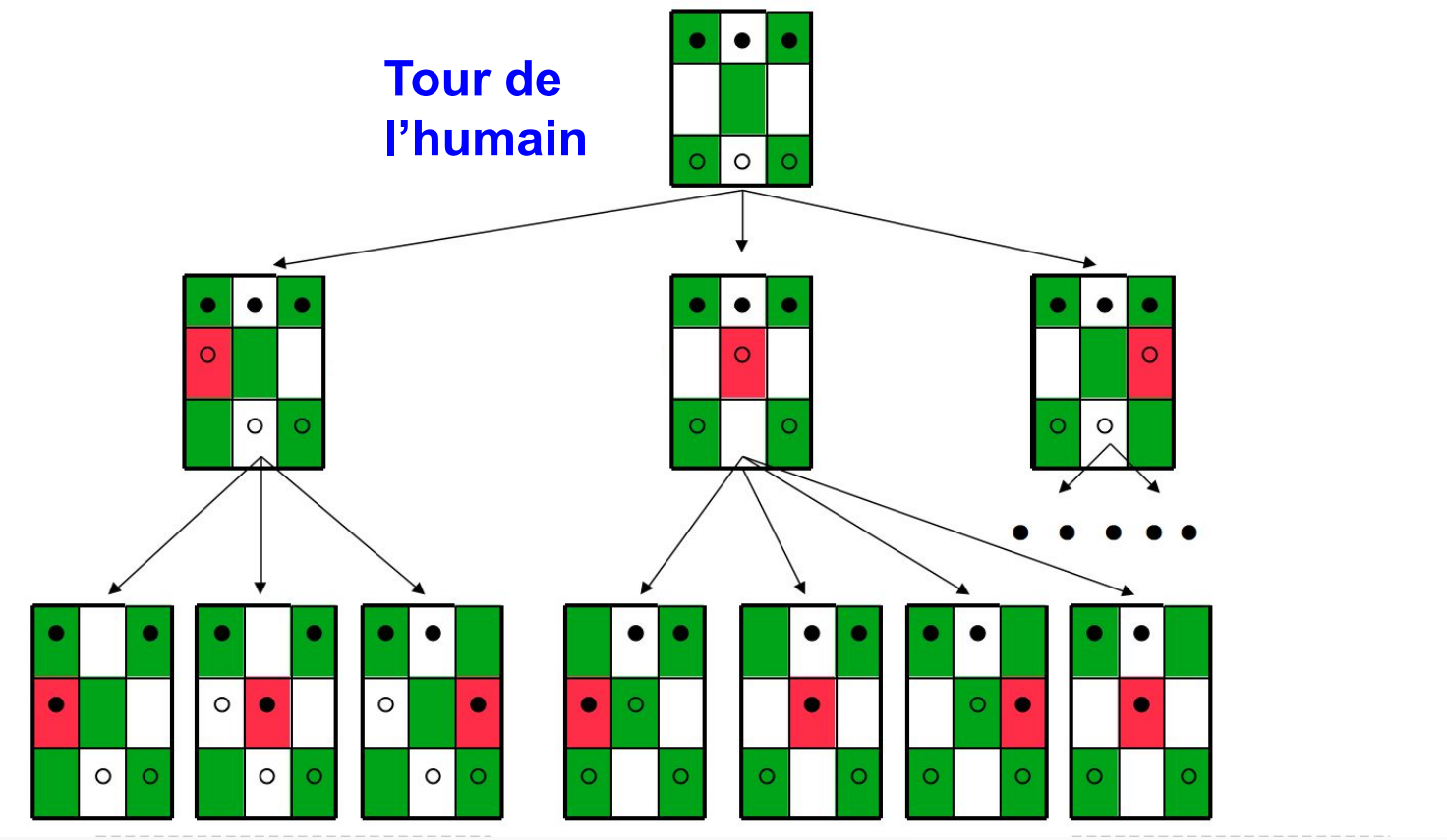
Plateau

Taille : 3x3

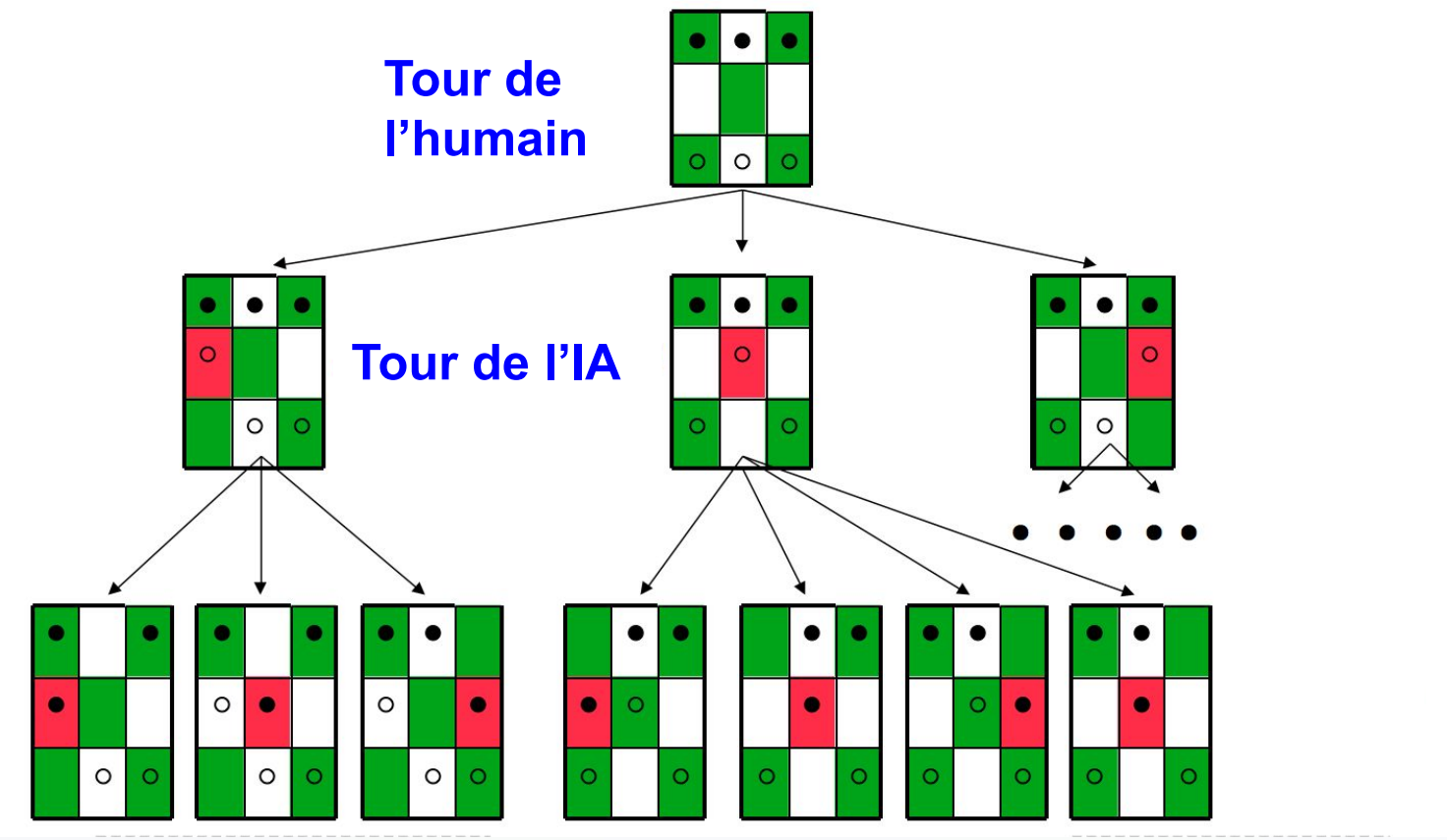


https://proceedings.mlr.press/v170/marx22a/marx22a.pdf

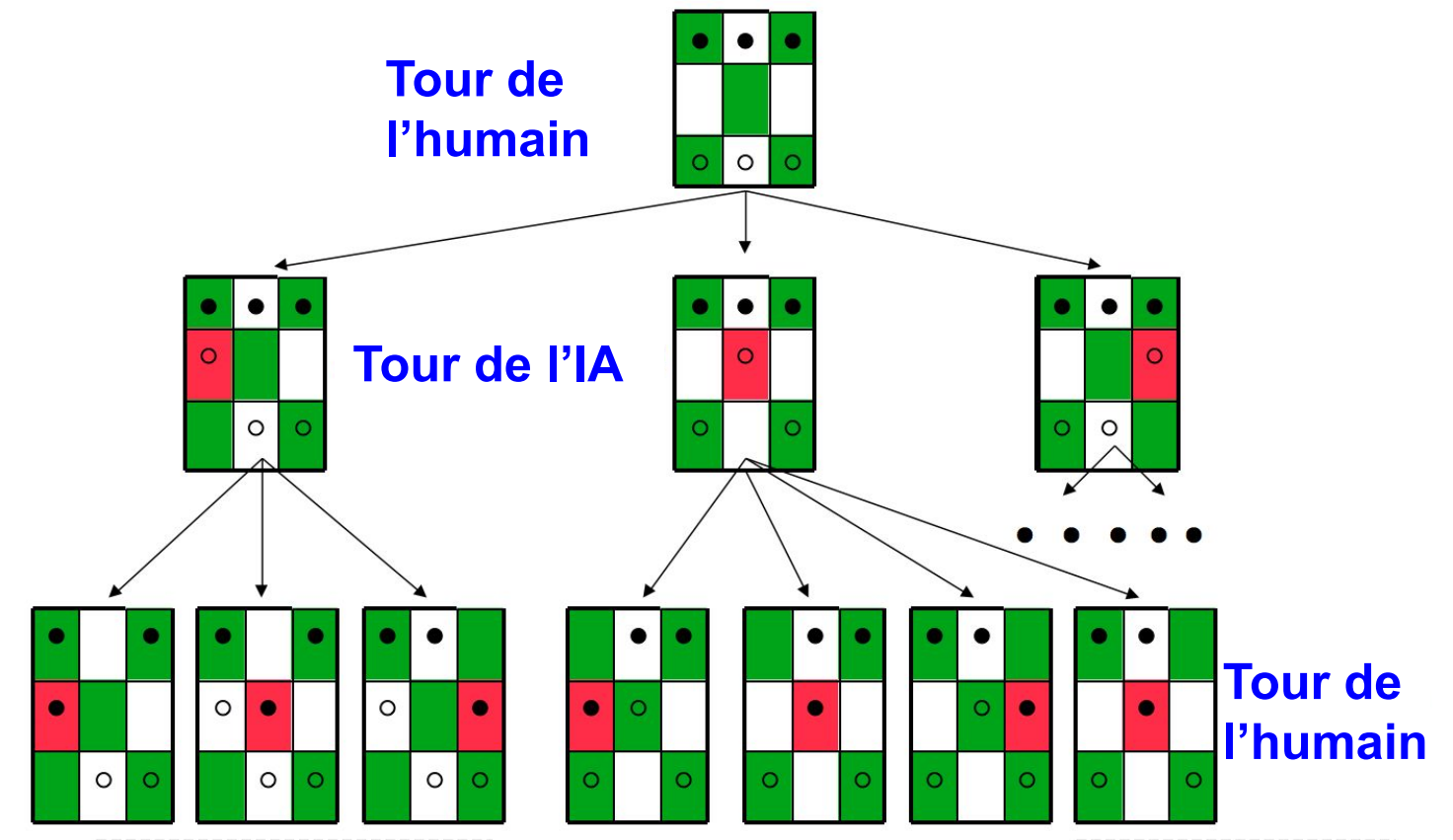
Règles



Règles

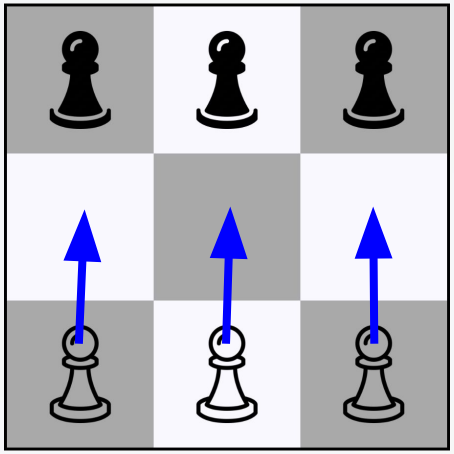


Règles

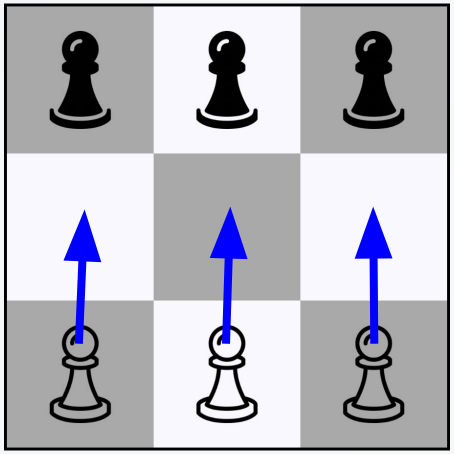


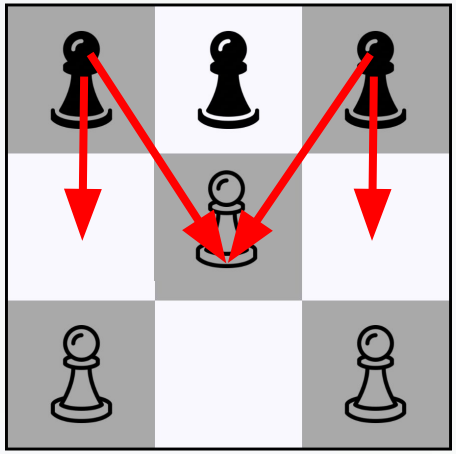
https://courses.grainger.illinois.edu/cs440/fa2019/Lectures/lect33.html

Règles

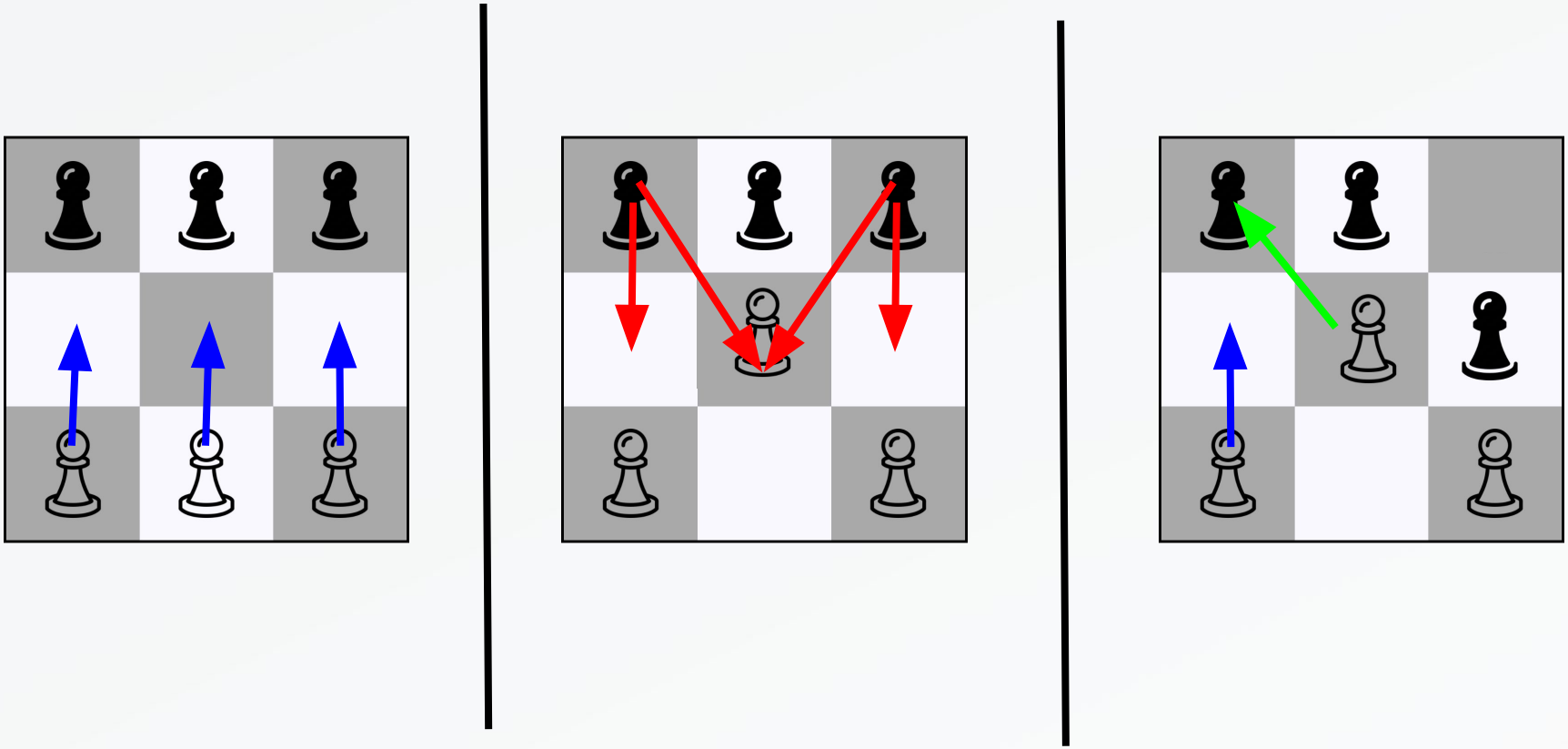


Règles





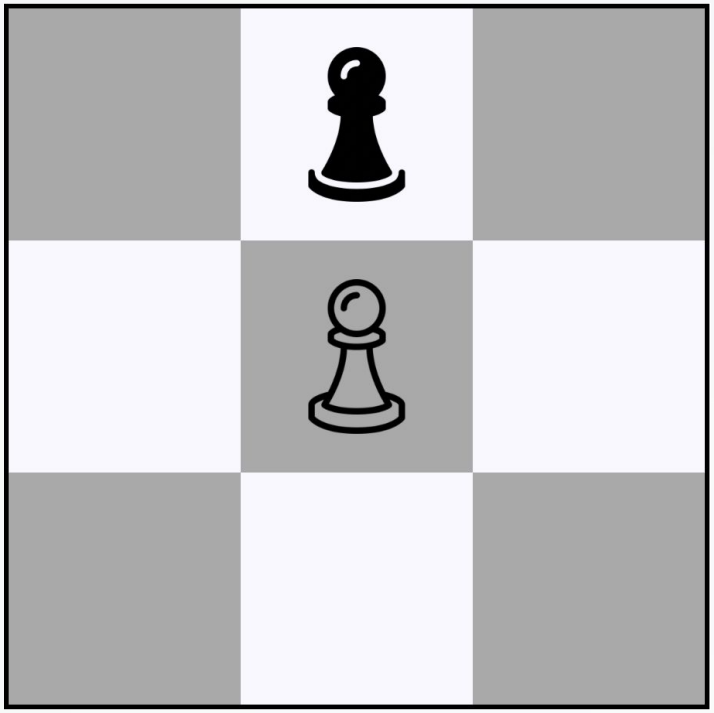
Règles



Règles

Qui gagne dans cette configuration ?

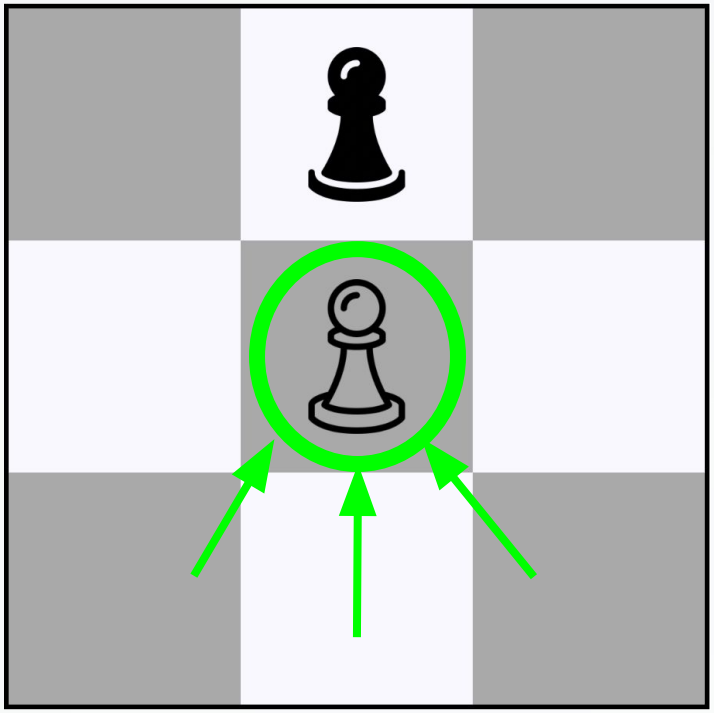
Tour de l'IA



Règles

Qui gagne dans cette configuration ?

Tour de l'IA



L'humain a bloqué l'IA. Il gagne

Fonctions

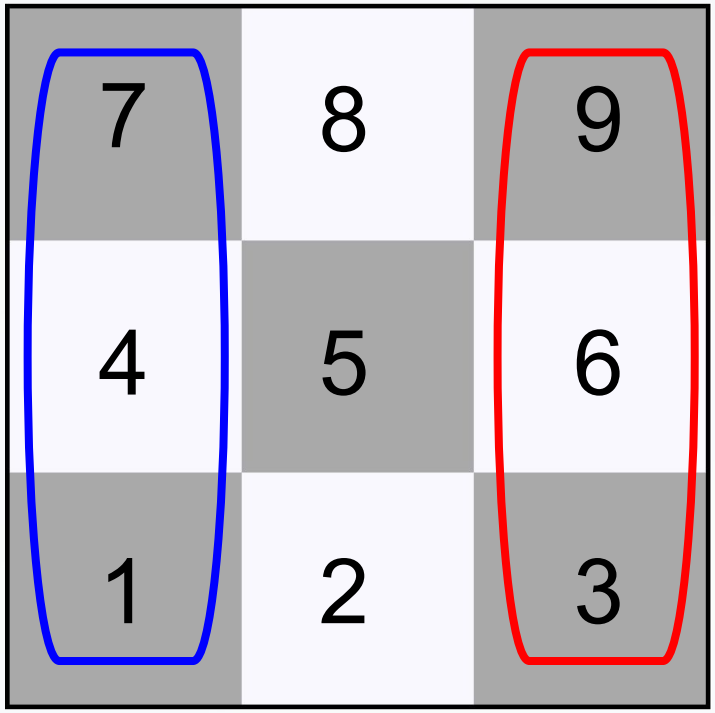
```
def surPlateau(n):
    return 1 <= n <= 9
```

7	8	9
4	5	6
1	2	3

Fonctions

```
def colDroite (n):
    return n % 3 == 0

def colGauche (n):
    return n % 3 == 1
```




```
n % 3 == 1
```

```
n % 3 == 0
```

Fonctions

```
position = [[1,2,3], [7,8,9]]
```

7	8	9
4	5	6
1	2	3

```
def key (position):
    [hu,ia] = position
    plateau = ""
    for k in [7,8,9,4,5,6,1,2,3]:
        if k in hu: plateau += 'O'
        elif k in ia: plateau += 'X'
        else: plateau += ' '
    return plateau
```

} chaîne de caractères

Fonctions

```
position = [[1,2,3], [7,8,9]]
```

7	8	9
4	5	6
1	2	3

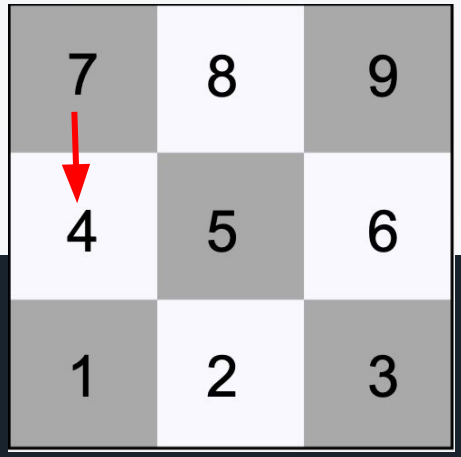
```
def key (position):
    [hu,ia] = position
    plateau = ""
    for k in [7,8,9,4,5,6,1,2,3]:
        if k in hu: plateau += 'O'
        elif k in ia: plateau += 'X'
        else: plateau += ' '
    return plateau
```

chaînes de caractères

```
In [2]: key(position = [[1,2,3], [7,8,9]])
Out[2]: 'XXX   OOO'

In [3]: hexapawn()
[[1, 2, 3], [7, 8, 9]]
XXX
...
OOO
```

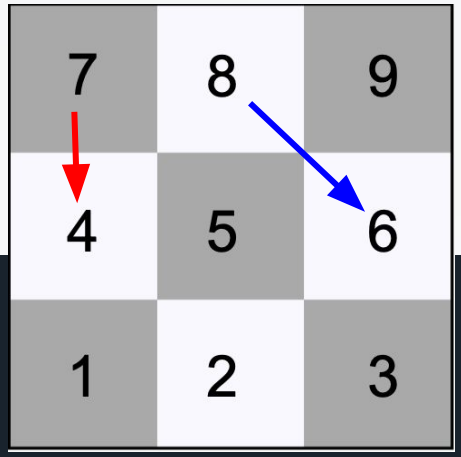
Fonctions



dep = 74

```
def jouables(position):
    [hu,ia]=position
    dep=[]
    for k in ia:
        avancer = k - 3
        if surPlateau (avancer) and avancer not in hu and avancer not in ia:
            dep += [10 * k + avancer]
        droite = k - 2
        if surPlateau (droite) and not (colDroite (k)) and droite in hu:
            dep += [10 * k + droite]
        gauche = k - 4
        if surPlateau (gauche) and not (colGauche (k)) and gauche in hu:
            dep += [10 * k + gauche]
    return dep
```

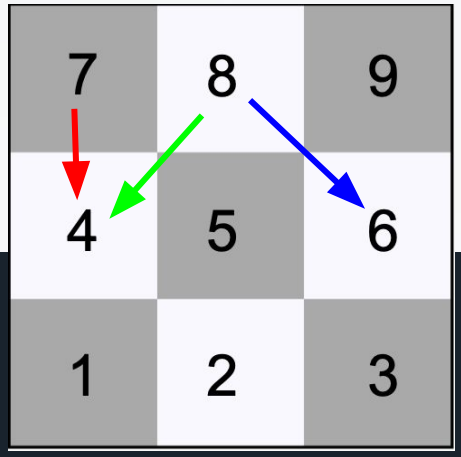
Fonctions



dep = 74 dep = 86

```
def jouables(position):
    [hu,ia]=position
    dep=[]
    for k in ia:
        avancer = k - 3
        if surPlateau (avancer) and avancer not in hu and avancer not in ia:
            dep += [10 * k + avancer]
        droite = k - 2
        if surPlateau (droite) and not (colDroite (k)) and droite in hu:
            dep += [10 * k + droite]
        gauche = k - 4
        if surPlateau (gauche) and not (colGauche (k)) and gauche in hu:
            dep += [10 * k + gauche]
    return dep
```

Fonctions



dep = 74 dep = 86 dep = 84

```
def jouables(position):
    [hu,ia]=position
    dep=[]
    for k in ia:
      avancer = k - 3
      if surPlateau (avancer) and avancer not in hu and avancer not in ia:
          dep += [10 * k + avancer]
      droite = k - 2
      if surPlateau (droite) and not (colDroite (k)) and droite in hu:
          dep += [10 * k + droite]
      gauche = k - 4
      if surPlateau (gauche) and not (colGauche (k)) and gauche in hu:
          dep += [10 * k + gauche]
    return dep
```

Exemple

```
In [11]: hexapawn()
[[1, 2, 3], [7, 8, 9]]
XXX
...
000
----------
Votre coup ou 0 ? 14
Je joue 14
[[2, 3, 4], [7, 8, 9]]
XXX
0..
.00
----------
Je joue 85
[[2, 3, 4], [7, 9, 5]]
X.X
0X.
.00
----------
['XXX0   00', 85]
Votre coup ou 0 ? 35
Je joue 35
[[2, 4, 5], [7, 9]]
X.X
00.
.0.
```

```
----------
Je joue 96
[[2, 4, 5], [7, 6]]
X..
00X
.0.
----------
['X X00  0 ', 96]
Votre coup ou 0 ? 58
Je joue 58
[[2, 4, 8], [6, 7]]
X0.
0.X
.0.
----------
Vous gagnez...
```

Explication/Fonctions

```
strategie = {}
```

```
{'XXX  000 ': [74, 86]}
```

stratégie

IA

Explication/Fonctions

```
strategie = {}
```

```
{'XXX  000 ': [74, 86]}
```

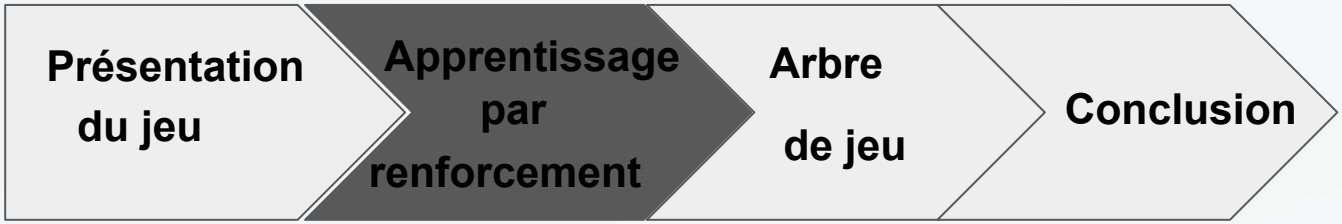
stratégie

IA

partie gagnée

victoire
ajouter son dernier coup
dans sa stratégie

```
if gagnante (position):
    print ("Je gagne...")
    strategie [historique [0]] += [historique [1]]
```



Explication/Fonctions

```
strategie = {}
```

```
{'XXX  000 ': [74, 86]}
```

stratégie

IA

partie gagnée

partie perdue

victoire
ajouter son dernier coup dans sa stratégie

défaite
retirer son dernier coup dans sa stratégie

```
if gagnante (position):
    print ("Je gagne...")
    strategie [historique [0]] += [historique [1]]
```

```
else:
    print( "Vous gagnez...")
    strategie[historique[0]].remove(historique[1])
```

Fonctions

```
strategie = {}
def hexapawn ():
    while True:
        position = [[1,2,3], [7,8,9]]
        affiche (position)
        while not (gagnante (position)):
            valide = False
            while not (valide):
                coup = input ("Votre coup ou 0 ? ")
                if coup == '0':
                    return
                valide = humain (coup, position)
                if valide:
                    position = jouer (coup, position, 0)
                else:
                    print ("Vous ne pouvez pas jouer ca !")
```

} position initiale

Fonctions

```
strategie = {}
def hexapawn ():
    while True:
        position = [[1,2,3], [7,8,9]]
        affiche (position)
        while not (gagnante (position)):
            valide = False
            while not (valide):
                coup = input ("Votre coup ou 0 ? ")
                if coup == '0':
                    return
                valide = humain (coup, position)
                if valide:
                    position = jouer (coup, position, 0)
                else:
                    print ("Vous ne pouvez pas jouer ca !")
```

} position initiale

tant que la position n'est pas gagnante
→ la partie continue

Fonctions

```
strategie = {}
def hexapawn ():
    while True:
        position = [[1,2,3], [7,8,9]]
        affiche (position)
        while not (gagnante (position)):
            valide = False
            while not (valide):
                coup = input ("Votre coup ou 0 ? ")
                if coup == '0':
                    return
                valide = humain (coup, position)
                if valide:
                    position = jouer (coup, position, 0)
                else:
                    print ("Vous ne pouvez pas jouer ca !")
```

} position initiale

tant que la position n'est pas gagnante
→ la partie continue

} entrer un coup (joueur hu)
sinon fin du jeu

Fonctions

```
strategie = {}
def hexapawn ():
    while True:
        position = [[1,2,3], [7,8,9]]
        affiche (position)
        while not (gagnante (position)):
            valide = False
            while not (valide):
                coup = input ("Votre coup ou 0 ? ")
                if coup == '0':
                    return
                valide = humain (coup, position)
                if valide:
                    position = jouer (coup, position, 0)
                else:
                    print ("Vous ne pouvez pas jouer ca !")
```

} position initiale

tant que la position n'est pas gagnante
→ la partie continue

} entrer un coup (joueur hu)
sinon fin du jeu

} si le coup est valide : jouer et modifier la position

Fonctions

```
if not(gagnante (position)):
    iacherche = rd.choice(jouables(position))
    historique = [key (position)]
    jouer (str (iacherche), position, 1)
    historique += [iacherche]
    print(historique)
    if gagnante (position):
        print ("Je gagne...")
        strategie [historique [0]] += [historique [1]]
else:
    print( "Vous gagnez...")
    strategie[historique[0]].remove(historique[1])
```

si la position n'est pas gagnante pour hu : cherche un coup jouable aléatoire pour l'ia, le jouer et l'afficher

Fonctions

```
if not(gagnante (position)):
    iacherche = rd.choice(jouables(position))
    historique = [key (position)]
    jouer (str (iacherche), position, 1)
    historique += [iacherche]
    print(historique)
    if gagnante (position):
        print ("Je gagne...")
        strategie [historique [0]] += [historique [1]]
else:
    print( "Vous gagnez...")
    strategie[historique[0]].remove(historique[1])
```

si la position n'est pas gagnante pour hu : cherche un coup jouable aléatoire pour l'ia, le jouer et l'afficher

si la position est gagnante pour l'ia : fin du jeu et on ajoute le dernier coup à stratégie

Fonctions

```
if not(gagnante (position)):
    iacherche = rd.choice(jouables(position))
    historique = [key (position)]
    jouer (str (iacherche), position, 1)
    historique += [iacherche]
    print(historique)
    if gagnante (position):
        print ("Je gagne...")
        strategie [historique [0]] += [historique [1]]
else:
    print( "Vous gagnez...")
    strategie[historique[0]].remove(historique[1])
```

si la position n'est pas gagnante pour hu : cherche un coup jouable aléatoire pour l'ia, le jouer et l'afficher

si la position est gagnante pour l'ia : fin du jeu et on ajoute le dernier coup à stratégie

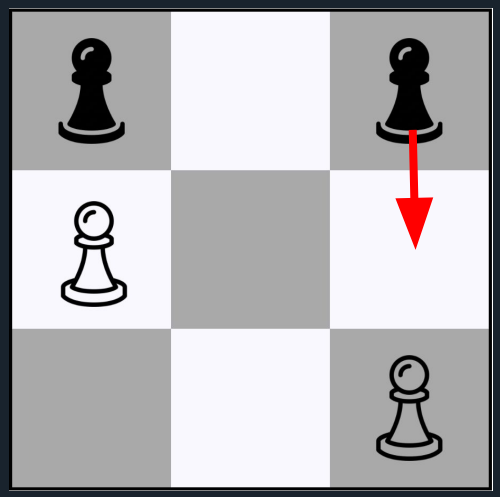
sinon hu gagne et on retire le coup précédent de l'ia dans stratégie

Exemple

```
In [13]: hexapawn()
partie 1 :  ia
partie 2 :  ia
partie 3 :  ia
partie 4 :  hu
partie 5 :  hu
partie 6 :  ia
partie 7 :  hu
partie 8 :  hu
partie 9 :  hu
partie 10 :  ia
partie 11 :  ia
partie 12 :  ia
partie 13 :  ia
partie 14 :  ia
partie 15 :  ia
partie 16 :  ia
partie 17 :  ia
partie 18 :  hu
partie 19 :  ia
partie 20 :  hu
Le score est : [7, 13]
{'XXX  OOO ': [74, 86], 'X X  OO  ': [74, 74], 'X X OXO  ': [63, 74, 75, 95, 63, 63], '  XOXX   ': [52, 63
52, 52], 'XXXO   OO': [84], 'X XOO  O ': [75, 95], 'X XO    O': [96, 96, 96], 'XX O O  O': [85, 86], 'XXX
O O': [75, 96, 95], ' XX XOO  ': [52, 51, 86], ' XXOX   O': [52, 53, 84, 96, 53], 'X XX O O ': [41, 42, 41
42, 42], ' XX O   O': [96, 95, 96, 96], ' XXO OO  ': [85, 86, 84], '  XOOO   ': []}
```

Exemple

```
In [13]: hexapawn()
partie 1 :  ia
partie 2 :  ia
partie 3 :  ia
partie 4 :  hu
partie 5 :  hu
partie 6 :  ia
partie 7 :  hu
partie 8 :  hu
partie 9 :  hu
partie 10 :  ia
partie 11 :  ia
partie 12 :  ia
partie 13 :  ia
partie 14 :  ia
partie 15 :  ia
partie 16 :  ia
partie 17 :  ia
partie 18 :  hu
partie 19 :  ia
partie 20 :  hu
Le score est : [7, 13]
{'XXX  000 ': [74, 86], 'X X  00  ': [74, 74], 'X X 0X0 ': [63, 74, 75, 95, 63, 63], '  X0XX   ': [52, 63
52, 52], 'XXX0   00': [84], 'X X00  0 ': [75, 95], 'X X0    0': [96, 96, 96], 'XX 0 0  0': [85, 86], 'XXX
0 0': [75, 96, 95], ' XX X00  ': [52, 51, 86], ' XX0X   0': [52, 53, 84, 96, 53], 'X XX 0 0 ': [41, 42, 41
42, 42], ' XX 0   0': [96, 95, 96, 96], ' XX0 00  ': [85, 86, 84], '  X000   ': []}
```



clé de la position

coups

Exemple

Stratégie après 100 parties

```
Le score est : [13, 87]
{'XXX 0 0 0': [75, 95], 'XX 0X   0': [52, 53, 84, 52], 'X  XX0   ': [41, 52, 52, 52], ' XX X00  ': [52,
51, 86, 51, 52, 52, 52], 'XXX  000 ': [86], 'XXX0   00': [84], 'XX  0   0': [74], ' X  X0   ': [52, 86,
52, 52], '  X0XX   ': [52, 63, 52], 'X XX 0 0 ': [41, 42, 42, 42, 42, 42, 42, 41, 42, 42, 42, 41, 41, 42]
'X X0 X 0 ': [63, 62, 62, 62, 62, 62, 62, 62, 62, 62], 'X X0    0': [96, 96, 96, 96, 96, 96, 96, 96, 96,
96, 96, 96, 96], 'X X 0X0  ': [63, 75, 95, 63, 63, 63], 'X XX0   0': [41, 75, 95, 41, 41, 41, 41, 41, 41,
41, 41], '  XX0    ': [41, 95, 95, 95], ' XX0X   0': [52, 53, 84, 96, 52], ' XX 0 0  ': [96], ' X 00X   '
[63, 84, 63, 63], 'XX  0 0  ': [74, 74, 74, 74, 74], 'X  0XX   ': [52, 63, 63, 63], 'X X  00  ': [74, 74,
74, 74, 74, 74, 74, 74, 74], ' X X00   ': [41, 41, 41, 41], '  X 0X   ': [63, 95, 63, 63], ' XX 0   0':
[96, 95, 96, 96, 96], 'XX  X00  ': [52, 51, 74, 86, 51, 52, 52], 'X   0X   ': [63, 74, 75, 63, 63, 63], '
XXX0   ': [41, 52, 52]}
```

Augmentation du plateau

2M+1	…	3M
M+1	…	2M
1	…	M

Augmentation du plateau

```
position = [list(range(1,M+1)), list(range(2*M+1,3*M+1))]
```

```
def surPlateau(n):
    return 1 <= n <= 3*M

def colDroite(n):
    return n % M == 0

def colGauche(n):
    return n % M == 1

def key(position):
    [hu, ia] = position
    plateau = ""
    for k in list(range(1,3*M+1)):
        if k in hu:
            plateau += 'O'
        elif k in ia:
            plateau += 'X'
        else:
            plateau += ' '
    return plateau
```

2M+1	…	3M
M+1	…	2M
1	…	M

Augmentation du plateau

Calcul de taux de victoires

Taille du plateau : M	Taux de victoires (IA) sur 50 parties	Taux de victoires (IA) sur 100 parties
3	80%	96%
5	62%	78%
10	52%	63%

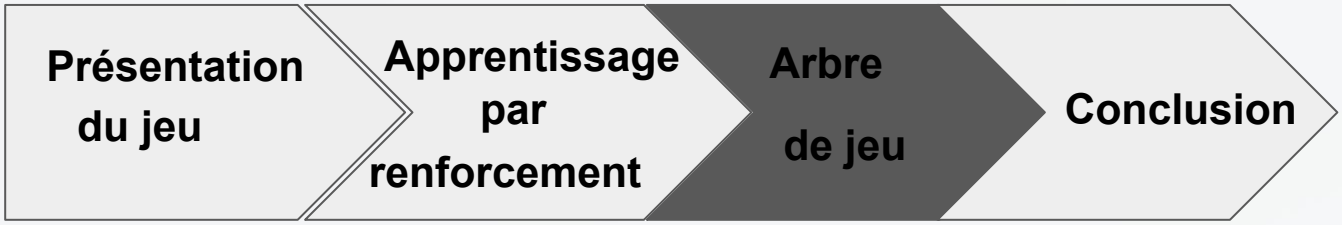
Augmentation du plateau

Calcul de taux de victoires

Taille du plateau : M	Taux de victoires (IA) sur 50 parties	Taux de victoires (IA) sur 100 parties
3	80%	96%
5	62%	78%
10	52%	63%

200 parties → 90%

600 parties → 90%



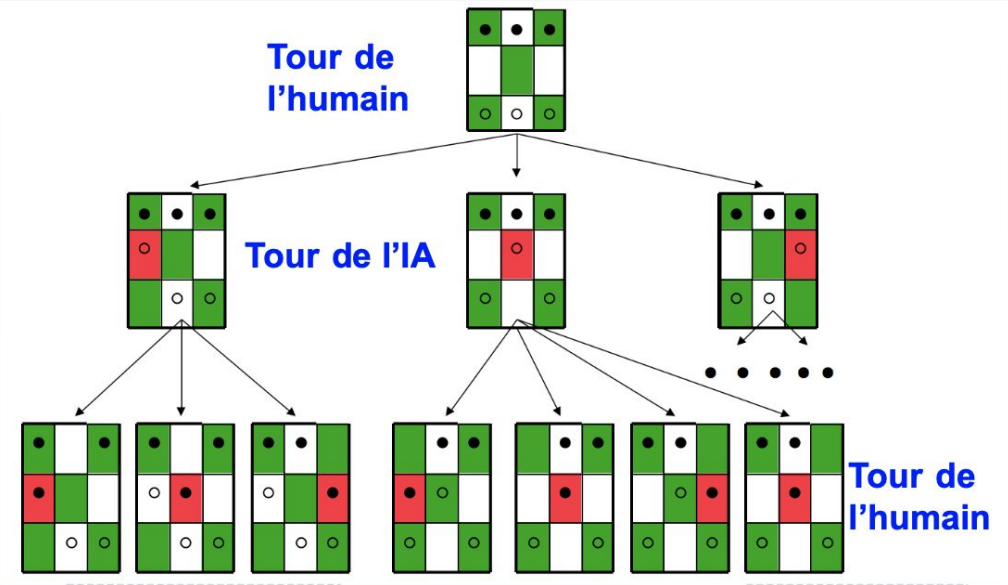
Explication/Fonctions

Existe-t-il une stratégie gagnante pour l'IA ?

Explication/Fonctions

Existe-t-il une stratégie gagnante pour l'IA ?

Arbre de jeu : représenter toutes les configurations possibles du jeu

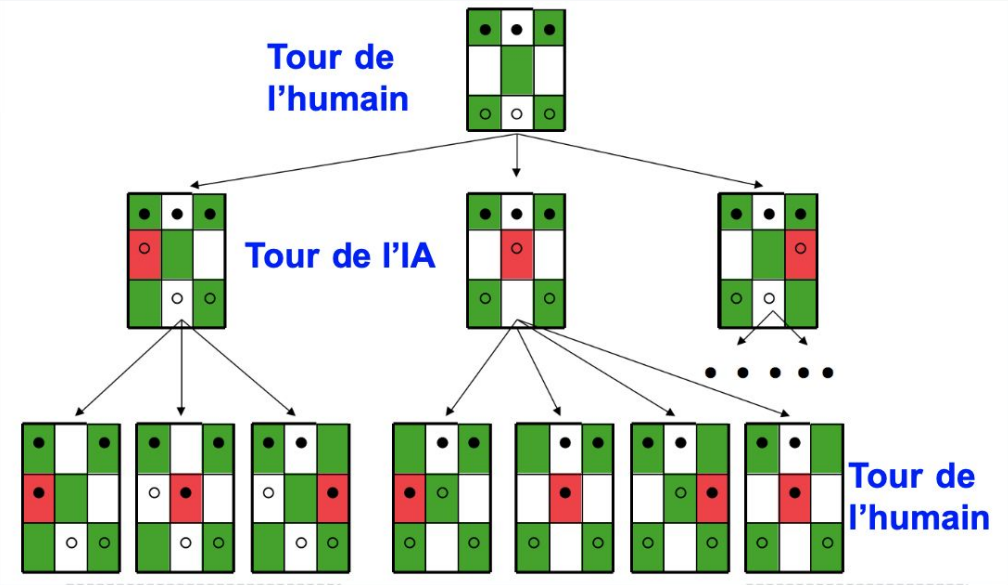


https://courses.grainger.illinois.edu/cs440/fa2019/Lectures/lect33.html

Explication/Fonctions

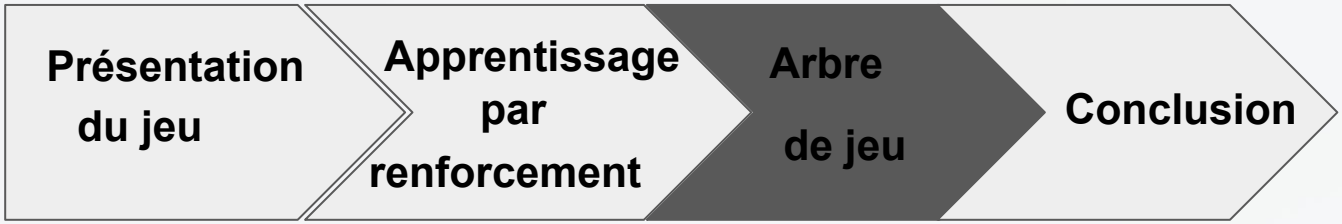
Existe-t-il une stratégie gagnante pour l'IA ?

Arbre de jeu : représenter toutes les configurations possibles du jeu



https://courses.grainger.illinois.edu/cs440/fa2019/Lectures/lect33.html

Algorithme min-max pour créer un dictionnaire 'clé position gagnante' : [coups gagnants pour l'IA]

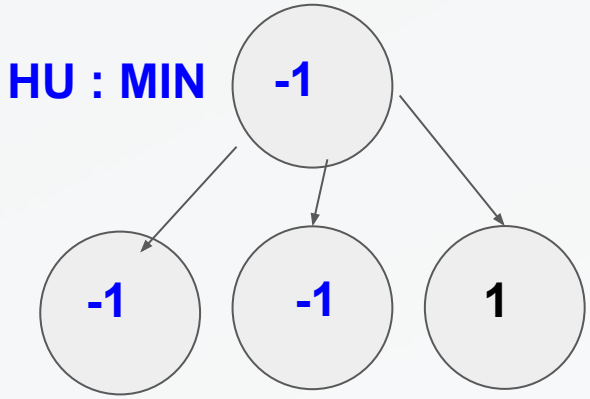


Explication/Fonctions

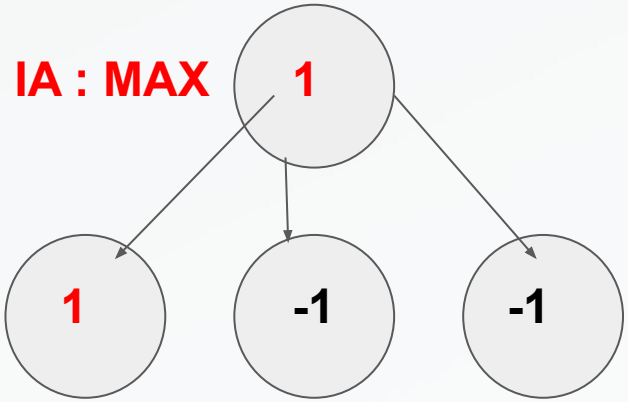
	HUMAIN gagne	IA gagne
SCORE d'1 POSITION GAGNANTE	**-1**	**1**

Explication/Fonctions

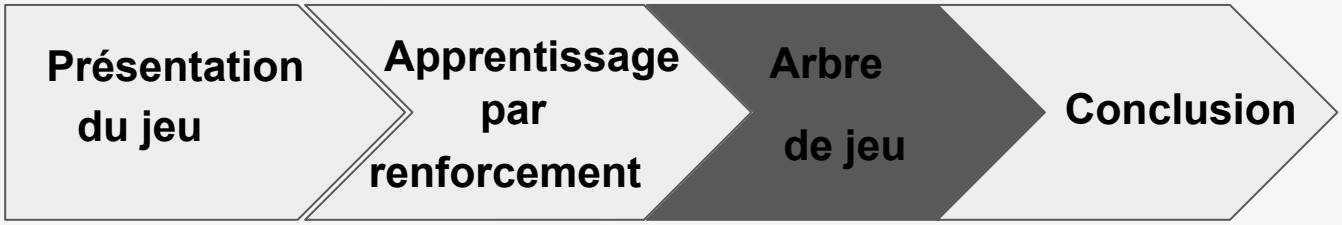
	HUMAIN gagne	IA gagne
SCORE d'1 POSITION GAGNANTE	-1	1



Minimum des scores : l'HU joue le coup qui minimise le score



Maximum des scores : l'IA joue le coup qui maximise le score



Explication/Fonctions

Mise en place de la stratégie de l'IA

Explication/Fonctions

Mise en place de la stratégie de l'IA

→ Utilisation de l'arbre pour trouver les coups gagnants

```
def coup_gagnant_ia(position):
    if key(position) in D:
        return D[key(position)][0]
```

Explication/Fonctions

Mise en place de la stratégie de l'IA

→ Utilisation de l'arbre pour trouver les coups gagnants

```
def coup_gagnant_ia(position):
    if key(position) in D:
        return D[key(position)][0]
```

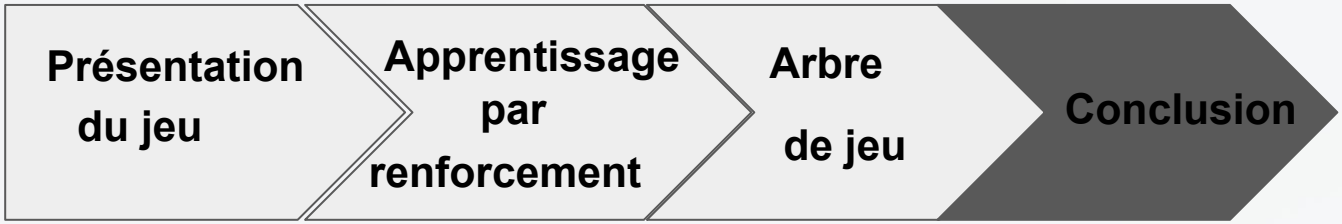
```
In [2]: D['XXX  OOO '][0]
Out[2]: 86
```

Explication/Fonctions

```
In [9]: D
Out[9]:
{'X XOO  O ': [95],
 'X XXO   O': [41],
 '  XXXO   ': [41],
 '  XXO    ': [41],
 'X  XXO   ': [41],
 'X  XO    ': [41],
 'X XO    O': [96],
 'X XX O O ': [41],
 'XXXO   OO': [84],
 'XX O O  O': [86],
 ' XXOX   O': [52],
 ' X OOX   ': [63],
 ' XX O   O': [96],
 ' X  XO   ': [52],
 ' XX XOO  ': [52],
 '  XOXX   ': [52],
 '  X OX   ': [63],
 ' XX O O  ': [96],
 ' X OX    ': [52],
 'XXX O O O': [75],
 'XX OX   O': [52],
 ' X XOO   ': [41],
 'XX  O   O': [74],
 'XX  XOO  ': [52],
 'X  OXX   ': [52],
 'X   OX   ': [63],
 'XX  O O  ': [74],
 ' XXO OO  ': [84],
 'X X OO O ': [75],
 'X XO X O ': [63],
 'X X OXO  ': [63],
 'X X  OO  ': [74],
 'XXX  OOO ': [86]}
```

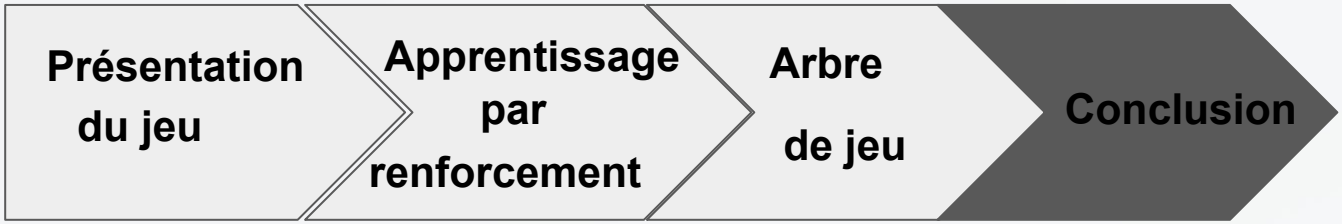
L'IA a 1 stratégie gagnante quelque soit les coups de l'HU

```
In [18]: hexapawn()
partie 1 :  ia
partie 2 :  ia
partie 3 :  ia
partie 4 :  ia
partie 5 :  ia
partie 6 :  ia
partie 7 :  ia
partie 8 :  ia
partie 9 :  ia
partie 10 :  ia
partie 11 :  ia
partie 12 :  ia
partie 13 :  ia
partie 14 :  ia
partie 15 :  ia
partie 16 :  ia
partie 17 :  ia
partie 18 :  ia
partie 19 :  ia
partie 20 :  ia
Le score est : [0, 20]
```



Conclusion

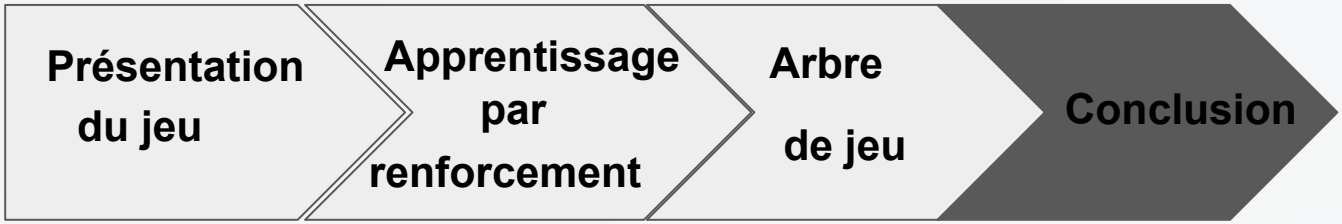
- **Les 2 méthodes mènent à la stratégie gagnante :**
  l'une directement et l'autre par renforcement (nécessite du temps).



Conclusion

- **Les 2 méthodes mènent à la stratégie gagnante :**
  l'une directement et l'autre par renforcement (nécessite du temps).

- **En augmentant le plateau, l'arbre de jeu n'est plus utilisable**
  (trop de configurations possibles à partir de M = 4).



Conclusion

- **Les 2 méthodes mènent à la stratégie gagnante :**
  l'une directement et l'autre par renforcement (nécessite du temps).

- **En augmentant le plateau, l'arbre de jeu n'est plus utilisable**
  (trop de configurations possibles à partir de M = 4).

- **L'apprentissage par renforcement reste possible pour des problèmes à hautes complexités, mais il prend plus de temps.**

Merci de m’avoir écouté

Avez-vous des questions ?

Explication/Fonctions

```
graphecomplet([[1,2,3], [7,8,9]],0))
```

```
D = {}
def graphecomplet(position, joueur):
    global D
    if gagnante(position,1-joueur)[0]:
        joueur_gagnant = gagnante(position,1-joueur)[1]
        if joueur_gagnant == 'hu':
            return [key(position),'hu',-1,[]]
        else:
            return [key(position),'ia',1,[]]
    else:
        coups = coups_possibles(position, joueur)
        if joueur == 0:
            L = [key(position),'hu',1]
            for coup in coups:
                nouvelle_position = jouer(str(coup), position, joueur)
                T = graphecomplet(nouvelle_position, 1-joueur)
                L.append(T)
                if T[2] == -1:
                    L[2] = -1
        if joueur == 1:
            L = [key(position),'ia',-1]
            for coup in coups:
                nouvelle_position = jouer(str(coup), position, joueur)
                T = graphecomplet(nouvelle_position, 1-joueur)
                L.append(T)
                if T[2] == 1:
                    L[2] = 1
                    if key(position) not in D:
                        D[key(position)] = [coup]
    return L
```

si la position est gagnante pour le joueur : note qui est le gagnant

Explication/Fonctions

```
graphecomplet([[1,2,3], [7,8,9]],0))
```

```
D = {}
def graphecomplet(position, joueur):
    global D
    if gagnante(position,1-joueur)[0]:
        joueur_gagnant = gagnante(position,1-joueur)[1]
        if joueur_gagnant == 'hu':
            return [key(position),'hu',-1,[]]
        else:
            return [key(position),'ia',1,[]]
    else:
        coups = coups_possibles(position, joueur)
        if joueur == 0:
            L = [key(position),'hu',1]
            for coup in coups:
                nouvelle_position = jouer(str(coup), position, joueur)
                T = graphecomplet(nouvelle_position, 1-joueur)
                L.append(T)
                if T[2] == -1:
                    L[2] = -1
        if joueur == 1:
            L = [key(position),'ia',-1]
            for coup in coups:
                nouvelle_position = jouer(str(coup), position, joueur)
                T = graphecomplet(nouvelle_position, 1-joueur)
                L.append(T)
                if T[2] == 1:
                    L[2] = 1
                    if key(position) not in D:
                        D[key(position)] = [coup]
    return L
```

si la position est gagnante pour le joueur : note qui est le gagnant

si hu gagne : position/score = -1

si ia gagne : position/score = 1

Explication/Fonctions

```
graphecomplet([[1,2,3], [7,8,9]],0))
```

```
D = {}
def graphecomplet(position, joueur):
    global D
    if gagnante(position,1-joueur)[0]:
        joueur_gagnant = gagnante(position,1-joueur)[1]
        if joueur_gagnant == 'hu':
            return [key(position),'hu',-1,[]]
        else:
            return [key(position),'ia',1,[]]
    else:
        coups = coups_possibles(position, joueur)
        if joueur == 0:
            L = [key(position),'hu',1]
            for coup in coups:
                nouvelle_position = jouer(str(coup), position, joueur)
                T = graphecomplet(nouvelle_position, 1-joueur)
                L.append(T)
                if T[2] == -1:
                    L[2] = -1
        if joueur == 1:
            L = [key(position),'ia',-1]
            for coup in coups:
                nouvelle_position = jouer(str(coup), position, joueur)
                T = graphecomplet(nouvelle_position, 1-joueur)
                L.append(T)
                if T[2] == 1:
                    L[2] = 1
                    if key(position) not in D:
                        D[key(position)] = [coup]
    return L
```

si la position est gagnante pour le joueur : note qui est le gagnant

si hu gagne : position/score = -1

si ia gagne : position/score = 1

joueur 0 : hu

joueur 1 : ia

Explication/Fonctions

```
graphecomplet([[1,2,3], [7,8,9]],0))
```

```
D = {}
def graphecomplet(position, joueur):
    global D
    if gagnante(position,1-joueur)[0]:
        joueur_gagnant = gagnante(position,1-joueur)[1]
        if joueur_gagnant == 'hu':
            return [key(position),'hu',-1,[]]
        else:
            return [key(position),'ia',1,[]]
    else:
        coups = coups_possibles(position, joueur)
        if joueur == 0:
            L = [key(position),'hu',1]
            for coup in coups:
                nouvelle_position = jouer(str(coup), position, joueur)
                T = graphecomplet(nouvelle_position, 1-joueur)
                L.append(T)
                if T[2] == -1:
                    L[2] = -1
        if joueur == 1:
            L = [key(position),'ia',-1]
            for coup in coups:
                nouvelle_position = jouer(str(coup), position, joueur)
                T = graphecomplet(nouvelle_position, 1-joueur)
                L.append(T)
                if T[2] == 1:
                    L[2] = 1
                    if key(position) not in D:
                        D[key(position)] = [coup]
    return L
```

si la position est gagnante pour le joueur : note qui est le gagnant

si hu gagne : position/score = -1

si ia gagne : position/score = 1

joueur 0 : hu

on regarde les coups possibles et pour chaque coup on fait un appel récursif de la fonction

joueur 1 : ia

Explication/Fonctions

```
graphecomplet([[1,2,3], [7,8,9]],0))
```

```
D = {}
def graphecomplet(position, joueur):
    global D
    if gagnante(position,1-joueur)[0]:
        joueur_gagnant = gagnante(position,1-joueur)[1]
        if joueur_gagnant == 'hu':
            return [key(position),'hu',-1,[]]
        else:
            return [key(position),'ia',1,[]]
    else:
        coups = coups_possibles(position, joueur)
        if joueur == 0:
            L = [key(position),'hu',1]
            for coup in coups:
                nouvelle_position = jouer(str(coup), position, joueur)
                T = graphecomplet(nouvelle_position, 1-joueur)
                L.append(T)
                if T[2] == -1:
                    L[2] = -1
        if joueur == 1:
            L = [key(position),'ia',-1]
            for coup in coups:
                nouvelle_position = jouer(str(coup), position, joueur)
                T = graphecomplet(nouvelle_position, 1-joueur)
                L.append(T)
                if T[2] == 1:
                    L[2] = 1
                    if key(position) not in D:
                        D[key(position)] = [coup]
    return L
```

si la position est gagnante pour le joueur : note qui est le gagnant

si hu gagne : position/score = -1

si ia gagne : position/score = 1

joueur 0 : hu

on regarde les coups possibles et pour chaque coup on fait un appel récursif de la fonction

joueur 1 : ia

on regarde les coups possibles pour l'ia et pour chaque coup on fait un appel récursif en vérifiant si le résultat est présent dans le dictionnaire